CSC380 - Homework 5 - Problem 3

All cells are marked with instructions to insert your code. Please complete all cells as directed.

What to turn in:

- Please print the notebook containing the answers and results into a pdf file (you can use `File – Print`). Submit this pdf file to the main homework entry in gradescope. Be sure to locate your answers for each problem when you submit, as ususal. In the worst case where you cannot print it into a pdf file somehow, you can create a Microsoft word document and then copy-paste screenshots showing your code and output parts by parts.
- You also need to submit this jupyter notebook file filled with your answers in the code entry in gradescope.

Description:

This homework will familiarize you with linear regression. You will be using the *Prostate Cancer Dataset* from a study by Stamey et al. (1989). The study aims to predict prostate-specific antigen levels from clinical measures in men about to receive a radical prostatectomy.

The data contain 8 features:

- log cancer volume (lcavol)
- log prostate weight (lweight)
- age (age)
- log amount of benign prostatic hyperplasia (lbph)
- seminal vesicle invasion (svi)
- log of capsular penetration (lcp)
- Gleason score (gleason)
- percent of Gleason scores 4 or 5 (pgg45)

The data use a fixed Train / Test split, which we will load below.

In [2]:
```
#All finalised needed imports
import pandas as pd
import itertools
from sklearn.model_selection import cross_val_score
from sklearn import linear_model
import matplotlib.pyplot as plt
import numpy as np
import warnings
# Suppress warnings
```

```
warnings.filterwarnings("ignore")
```

In [3]:
```
df_train = pd.read_csv('prostate_train.csv')
df_train.head()
```

Out[3]:

	lcavol	lweight	age	lbph	svi	lcp	gleason	pgg45	lpsa
0	-0.579818	2.769459	50	-1.386294	0	-1.386294	6	0	-0.430783
1	-0.994252	3.319626	58	-1.386294	0	-1.386294	6	0	-0.162519
2	-0.510826	2.691243	74	-1.386294	0	-1.386294	7	20	-0.162519
3	-1.203973	3.282789	58	-1.386294	0	-1.386294	6	0	-0.162519
4	0.751416	3.432373	62	-1.386294	0	-1.386294	6	0	0.371564

Problem 1: Your First Regression

(5 points) We will begin by fitting our first ordinary least squares regression model. But first we need to do a little data management. You will notice that the data exist in a single data frame (one for Train and one for Test). The last column of the data frame ('lpsa') is the quantity that we wish to predict (the Y-value).

(a)

Do the following in the cell below,

- Create X_train and Y_train by separating out the last column ('lpsa') and store it in Y_train
- Do the same for X_test and Y_test
- Display the DataFrame X_train

In [4]:
```
features = ['lcavol', 'lweight', 'age', 'lbph', 'svi', 'lcp', 'gleason', 'pg
output = ['lpsa']
X_train = df_train[features]
Y_train = df_train['lpsa']

# Display training inputs
# Insert code here
X_train
```

Out[4]:

	lcavol	lweight	age	lbph	svi	lcp	gleason	pgg45
0	-0.579818	2.769459	50	-1.386294	0	-1.386294	6	0
1	-0.994252	3.319626	58	-1.386294	0	-1.386294	6	0
2	-0.510826	2.691243	74	-1.386294	0	-1.386294	7	20
3	-1.203973	3.282789	58	-1.386294	0	-1.386294	6	0
4	0.751416	3.432373	62	-1.386294	0	-1.386294	6	0
...	...	...	...	...	...	...	...	...
62	3.246491	4.101817	68	-1.386294	0	-1.386294	6	0
63	2.532903	3.677566	61	1.348073	1	-1.386294	7	15
64	2.830268	3.876396	68	-1.386294	1	1.321756	7	60
65	3.821004	3.896909	44	-1.386294	1	2.169054	7	40
66	2.882564	3.773910	68	1.558145	1	1.558145	7	80

67 rows × 8 columns

(b)

(5 points) Now we will fit our first model using a single feature ('lcavol'). Do the following in the cell below,

- Train a linear regression model on the 'lcavol' feature
- Compute the R-squared score of the model on the training data
- Scatterplot the training data for the 'lcavol' feature
- Plot the regression line over the scatterplot
- Label the plot axis / title and report the R-squared score

A couple of notes:

- Scikit-learn gets cranky when you pass in single features. In some versions you will need to use, X_train['lcavol'].values.reshape(-1, 1)
- To plot the regression line you can create a dense grid of points using numpy.arange, between the min() and max() of the feature values.

Documentation - Scikit-Learn - LinearRegression

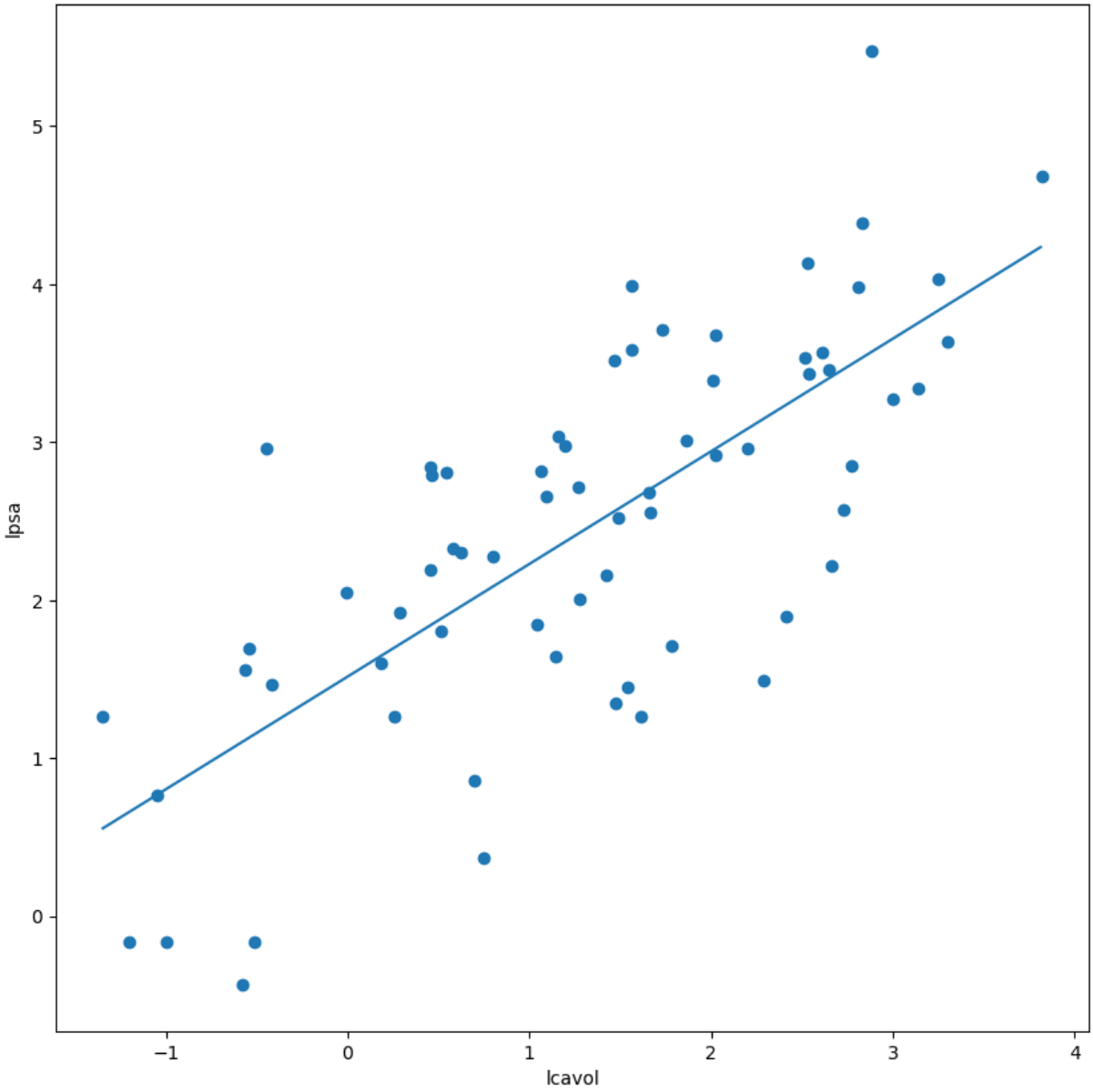
In [6]:
```
# Fit one feature
# Insert code here
# Train a linear regression model on the 'lcavol' feature
one_feat = linear_model.LinearRegression()
lcavol_XTrain = X_train[['lcavol']]
one_feat.fit(lcavol_XTrain, Y_train)
```

```
# Compute the R-squared score of the model on the training data
rSquared = one_feat.score(lcavol_XTrain, Y_train)

# Scatterplot the training data for the 'lcavol' feature
# plot
# Insert code here
minX = X_train['lcavol'].min()
maxX = X_train['lcavol'].max()
gridX = np.arange(minX, maxX, 0.01).reshape(-1, 1)

gridY = one_feat.predict(gridX)

plt.figure(figsize=(10, 10))
plt.scatter(X_train['lcavol'], Y_train, label='Training Set')
plt.plot(gridX, gridY, label='Regression Line')
plt.xlabel('lcavol')
plt.ylabel('lpsa')
plt.show()
```



Problem 2: Best Subset Feature Selection

Now we will look at finding the best subset of features out of all possible subsets. To do this, you will implement the Best Feature Subset Selection as presented in lecture (see lecture slides). We will break this into subproblems to walk through it. To help you with this we have provided a function findsubsets(S,k). When passed a set S this function will return a set of all subsets of size k, which you can iterate through to train models.

In [ ]:
```
def findsubsets(S,k):
    return set(itertools.combinations(S, k))
```

(a)

(8 points) We will start by getting familiar with the findsubsets() function. The variable 'features' was defined previously as a set of all feature names. In the cell do the following:

- Use findsubsets to find all possible subsets of 3 features
- Perform 5-fold cross validation to train a LinearRegression model on each set of 3 features
- Find the model with the highest average R^2 score (scoring='r2')
- Report the best performing set of features and the corresponding R^2 score

Documentation - Scikit-Learn - cross_val_score

In [ ]:
```
# Insert code here
```

(b)

(12 points) Now, repeat the above process for all subsets of all sizes. For each $k = 1, \ldots, 8$ find all possible subsets of k features and evaluate a model on each set of features using 5-fold cross validation. Report your findings as follows,

- Produce a scatterplot of R^2 values for every run with subset size on the horizontal axis, and R^2 on the vertical axis (label your plot axes/title)
- Find the best performing model overall and report the R^2 and features for that model

In [ ]:
```
# Insert code here
```

Excellent You have found the best set of features by brute-force search over all possible features. Good work.

Problem 3 : Ridge Regression

(a)

(3 points) The problem with brute force search over features is that it doesn't scale well. We can do it for 8 features, but we can't do it for larger sets of features. Instead, we will look at a simpler model selection strategy by using L2 regularized linear regression (a.k.a. Ridge Regression). Do the following in the cell below,

- Learn a Ridge regression model on training data with alpha=0.5
- Report the learned feature weights using the provided printFeatureWeights function

Documentation - Scikit-Learn - linear_model.Ridge

In [ ]:
```
def printFeatureWeights(f, w):
  for idx in range(len(f)):
    print('%s : %f' % (f[idx], w[idx]))

# Insert code here
```

(b)

(7 points) We chose the regularization coefficient alpha=0.5 somewhat arbitrarily. We now need to perform model selection in order to learn the best value of alpha. We will do that by using cross_val_score over a range of values for alpha. When searching for regularization parameters it is generally good practice to search in log-domain, rather than linear domain. For example, we will search in the range $[10^{-1}, 10^{3}]$. Using Numpy's "logspace" function this corresponds to the range $[-1, 3]$ in log-domain. In the cell below do the following,

- Create a range of 50 alpha values spaced logarithmically in the range $[10^{-1}, 10^{3}]$
- Perform 5-fold cross-validation of Ridge regression model for each alpha and record R^2 score for each run (there will be 5x50 values)
- Report the best R^2 score and the value of alpha that achieves that score
- Use Matplotlib errorbar() function to plot the average R^2 with 1 standard deviation error bars for each of the 50 alpha values

Documentation - Matplotlib - errorbar

Documentation - Numpy - logspace

In [ ]:
```
# Insert code here
```

Now that we have a good model we will look at what it has learned. Train the Ridge regression model using the selected alpha from the previous cell. Report the learned feature weights using the printFeatureWeights() function previously provided.

Problem 4 : LASSO

Ridge regression performs shrinkage of the weights using the L2 norm. This will drive some weights *close* to zero, but not exactly zero. The LASSO method replaces the L2 penalty with an L1 penalty. Due to properties of L1 discussed in lecture, this has the effect of learning exactly zero weights on some features when it is supported by the data. In this problem we will repeat procedure of learning a Ridge regression model, but we will instead use LASSO. Let's start by fitting a LASSO model with a fixed alpha value.

(a)

(4 points) In the cell below do the following,

- Fit LASSO with alpha=0.1
- Use printFeatureWeights() to report the learned feature weights

Documentation - Scikit-Learn - linear_model.Lasso

In [ ]:
```
# Insert code here
```

(b)

(6 points) Now we will find a good value of alpha using cross-validation. Due to differences in how the LASSO model is optimized, there are dedicated methods for performing cross-validation on LASSO. Scikit-Learn's LassoLarsCV class performs LASSO-specific cross-validation using an optimized Least Angle Regression (LARS) algorithm. In the cell below do the following,

- Using LassoLarsCV perform 20-fold cross validation to solve all solution paths for Lasso
- Plot mean +/- standard error of **mean squared error** versus regularization coefficient α
- Title the plot and axes
- Report the best alpha value and the corresponding average mean squared error from cross-validation

Note: LassoLarsCV returns mean squared error, rather than R^2. It also determines the set of α values automatically, which are stored in the cv_alphas_ attribute.

Documentation - Scikit-Learn - LassoLarsCV

In [ ]:
```
# Insert code here
```

Problem 5 : Evaluate on Test

In this problem we will train all of the best performing models chosen by Best Subsets, Ridge Regression, and LASSO. We will evaluate and compare these models on the test data. This dataset uses a standard train / test split so we begin by loading test data below.

In [ ]:
```
df_test = pd.read_csv('prostate_test.csv')
df_test.head()
```

(a)

(4 points) Recall that all of the data are stored in a single table, with the final column being the output 'lpsa'. Before evaluating on test you must first create X_test and Y_test input/outputs where Y_test is the final column of the DataFrame, and X_test contains all other columns.

In [ ]:
```
# Insert code here
```

(b) Best Subsets

(4 points) In Problem 2 you found the best subset of features for an ordinary least squares regression model by enumerating all feature subsets. Using the best selected features train the model below and report mean squared error on the test set.

Documentation - Scikit-Learn - MeanSquaredError

In [ ]:
```
# Insert code here
```

(c) Ridge Regression

(4 points) In the cell below, train a Ridge Regression model using the optimal regularization coefficient (α) found in Problem 2. Report mean squared error on the test set.

In [ ]:
```
# Insert code here
```

(d) LASSO Regression

(4 points) Now, train and evaluate your final model. Train a Lasso regression using the optimal α parameters from Problem 3 and report MSE on the test set.

In [ ]:
```
# Insert code here
```

(e) Compare feature weights for each model

(4 points) Now let's compare the feature weight learned by each of the three models. In

the cell below, report the regression weights for each feature under Best Subset, Ridge, and Lasso models evaluated above. To make the output easier to read, please use a Pandas DataFrame to display the data. To do this, create a Pandas DataFrame where each column contains regression weights for one of the previous models, and then display that DataFrame in the standard fashion. You should also provide feature names on each of the rows.

Documentation - Pandas - DataFrame

In [ ]:
```
# Insert code here
```